

doc-ai-kit

A framework for AI-led document processing at scale

September 2026

The job

You have a set of documents and the same questions to answer about each one, and there are too many of them for an analyst to read.

Claude can read the documents and answer those questions well enough to be useful. Even so, a substantial amount of work remains for the analyst who must:

- Choose a subset of documents and divide it into training, validation and test sets.
- Plan the labeling, then do it.
- Write the code that reads the documents and pulls out their text.
- Design the prompts, and write the code that sends them with the document text.
- Measure how well the answers do, then change the prompts and measure again.
- Write the code that runs the final prompts over the whole corpus.
- Collect the results and write a report.

None of it is difficult on its own. It is a week or two of setup before anyone sees a number, and it is rebuilt from scratch on the next project.

Most of that can be handed to an agent

An agent can do this work, given the right tools and the right instructions. doc-ai-kit supplies both.

The tools

A Python package, installed with pip at a fixed version. It provides the commands for each step: extracting text, drawing the sets, building the labeling sheet, scoring, tuning, running the corpus, writing the report.

A second command, `newproject`, creates a project folder with the settings, the file where the questions are declared, and somewhere for project-specific code.

The instructions

The new project also contains written guidance for each step: what it is for, what to run, what the output means, what to decide, and what comes next.

A Claude Code session started in the project folder reads that guidance and works through the project with you.

The commands work on their own as well. The guidance tells the agent how to use them, and where to stop and ask.

What the agent does with you

- Tells you where to put your documents and how to organise them.
- Helps you settle the questions to be answered and the form each answer takes.
- Writes the configuration and the first version of the prompts.
- Draws the training, validation and test sets, and explains what each is for.
- Produces a spreadsheet for you to label, and reads it back once you have filled it in.
- Checks for answers that appear too rarely to measure, and talks through the options.
- Runs the prompt tuning, one experiment at a time, within the limits you set.
- Applies the best prompt to every document and produces the answers.
- Writes the final report, including how accurate each answer is.

The labeling stays with you. Everything else is a conversation.

Inside a project

What you and the agent fill in

- `tasks.yaml` : the questions, the form of each answer, and how answers are compared
- `config.yaml` : the model, the limits, the spend ceiling
- `custom/` : project code, registered rather than patched into the package
- `data/pdfs/` : the documents

What the project accumulates

- `data/text/` : the extracted text, read by every later step
- `data/labels.xlsx` and `labels.jsonl` : your labels, before and after checking
- `data/splits.json` : which documents are training, validation and held back
- `runs/` : every scoring run, the experiment leaderboard, the spend
- `decisions.md` : each decision you made, with the numbers behind it
- `REPORT.md` and `PROMPT.md` : written at the end

The path a document takes

```
pdfs -> text cache -> DSPy program -> typed answers -> normalize -> compare
                        ^                                     |
                        |                                     v
                    optimizer <----- score <----- human labels
```

- Text is extracted once and cached. Tuning and production read the same text, because accuracy measured on one version of a document does not carry over to another.
- Answers come back typed: a date as a date, a number with its unit, a list as a list.
- Normalizing before comparing is what makes "\$1.25M" and "1,250,000 USD" the same answer, and "Texas" and "TX" the same answer.
- The score is what the optimizer uses to choose between one prompt and another.

Nine kinds of question

KIND	EXAMPLE	HOW AN ANSWER IS JUDGED
Yes or no	Is liability capped?	Same answer, reading yes/no loosely
One of a list	Which category of document is it?	Same value, after mapping synonyms to one spelling
Several of a list	Which risk flags apply?	The two sets compared
Exact text	The reference number	Equal after tidying spacing and case
Close-enough text	The organisation's name	Similar enough, above a threshold the project sets
A list of values	Who signed it?	Each answer paired with its best match; extras and misses counted
A number with a unit	The total amount, or a notice period	Same unit, within the project's tolerance
A date	The date it takes effect	Equal at the precision the document gave
A passage	The paragraph that states the obligation	Enough overlap with the labeled passage

The kind of question sets the comparison, the type of the answer, and the numbers the report can show. A fixed-choice question asked as free text loses the per-answer breakdown.

Scoring

- The label and the model's answer pass through the same cleanup code before they are compared, so the comparison cannot favour either side.
- A comparison returns counts, not a verdict: correct answers, wrong answers, missed answers, correct blanks. The report needs those separately to show how often the program is right apart from how much it finds.
- A wrong answer counts twice, as something got wrong and something missed.
- A blank against a blank label counts as correct. Most documents do not answer every question.
- A reply that cannot be parsed counts as a failure rather than a blank. It is retried, and by default the run stops rather than report numbers if it still fails.
- The comparison code has its own test set of near-misses, and nothing downstream runs until those pass.

Three sets of documents

Training

Supplies the worked examples that go into the prompt.

Validation

Used to compare versions of the prompt and choose one.

Held back

Scored once, at the end. That number is the one reported.

- The held-back documents are chosen before any model runs, from a seed recorded in the project.
- They are labeled without looking at model output. Labels made by correcting the model agree with the model, and any number measured against them comes out too high.
- They are scored once. A second scoring needs an explicit override, and the override is printed in the report.
- Questions with too few examples among them are reported as unmeasured rather than given a number.

Labeling

Labels come from a spreadsheet: one tab, one row per file, one column per question.

- `sample-labels` chooses which documents to label, and which of them are held back, before any model runs.
- `label-sheet` writes the spreadsheet. Rows can arrive filled in with the model's answers for correction, which is two to three times faster, or empty. The labeler chooses. Held-back rows are always empty.
- Answers are written the way people write them: `$1.25M`, `June 15, 2024`, `30 days`. A passage is labeled by pasting the sentence, and doc-ai-kit finds its position in the text.
- `import-labels` reads every cell with the scoring code, reports all problems at once, and writes nothing until they are fixed. It also warns when one cell holds two entries, which cannot be matched and lowers that question's ceiling.

How tuning works

The model is not retrained. What changes is the message it receives: some instructions, the questions from `tasks.yaml`, sometimes a few worked examples, then the document. DSPy, the open-source library doing the tuning, tries versions of that message, scores each one on the validation set, and keeps the best.

BootstrapFewShot

Runs the program, keeps the documents it answered entirely correctly, and puts a few in the prompt as examples. Every answer category gets at least one.

MIPROv2

Has a model draft alternative instructions, then tries combinations of instructions and examples, spending more attempts on the ones that score well.

GEPA

Reads the mistakes, including the question, the expected answer and what came back, and has a second model rewrite the instructions.

Each experiment changes one thing and records its score per question, its cost and its runtime. The best so far is pinned, and the next experiment starts from it.

Limits enforced in code

LIMIT	WHAT IT PREVENTS
The held-back set locks after one scoring	Scoring twice and keeping the better number
Labels and splits are read-only during tuning	A run editing the thing it is measured against
A set number of experiments and model calls	A search that continues until the validation set gives way
A spend ceiling, checked before each paid step	Learning the cost after the fact. Spend is recorded per step
Tuning requires written labeling rules	A question whose rule was never written down, and so cannot be labeled consistently
Replies are parsed strictly	An unreadable reply counting as a blank

Each refusal says what to change. Limits can be raised, but only as a decision recorded in the project's log.

Traceability and repeatability

Every run records what produced it

- the settings: optimizer, model, call limits, the size of each set
- the one thing that experiment changed, named by the person who ran it
- which experiment it started from
- its score per question, its cost and its runtime
- the package version and a fingerprint of the settings
- the prompt itself, loaded unchanged for the full run

A leaderboard holds one row per experiment. `decisions.md` holds the choices a person made, with the numbers they had at the time.

The model itself is not repeatable: the same prompt can come back worded differently. The procedure, the sets and the scoring are fixed, which is what makes a difference measurable.

The process is the same every project

- the same commands in the same order, from one package version
- the sets drawn from a recorded seed, so the division can be reproduced
- text extracted once and cached, so every later step reads the same words
- answers saved from every run, so scoring can be corrected and repeated without calling the model again
- one row per finished project in a shared ledger

Take any number in the report, open the run behind it, and the prompt, the settings and the documents are all there.

What the report shows

- Per-question numbers appear next to the average in every run, because the average on its own hides a question that has collapsed.
- Yes/no questions are reported on their yes answers, which is what the headline number measures. Counting both answers together raises it: a program that finds 6 of the 14 documents where the answer is yes, and answers no correctly everywhere else, reads 0.75 either way when both answers are pooled, against 1.00 precision and 0.43 recall on the yes answers.
- Every number carries the range it could sit in. Ten documents, all correct, still allow a true error rate near 25%.
- The error file lists three mistakes per question. Given the full list, the optimizer writes rules about individual documents that do not hold on new ones.
- Borderline matches are listed for review, raw and normalized, because no threshold separates a typo from a different company.
- Predictions are saved, so correcting the comparison code and re-scoring costs nothing.

Scanned pages

A scanned page has no text layer, so any question answered from it fails, and no change to the prompt affects that.

- Pages are judged one at a time. A page with almost no text, where images cover most of it, is sent to Claude as an image, and the transcription replaces that page in the text cache.
- Judging a document on its average misses this. A report with twenty typed pages and five scanned pages of accounts averages out fine, and those five pages come through empty.
- Tables come back row by row, so a figure stays in its row. About 3,900 input tokens per page. Each page is cached, so re-extracting does not pay for it again.
- Pages that still have no text are counted in the manifest, and those documents are sent for review.

Running the full corpus

How it runs

Production goes through Anthropic's batch service at half the price. The requests are captured from the normal path at the point of sending, so a batch request is the same request, and replies are parsed by the same code.

Measured on a test corpus: 48 of 48 requests succeeded, input tokens matched the live run exactly, accuracy was 0.996 either way, and the cost went from \$0.238 to \$0.119.

Batch identifiers are written down before any waiting, and each answer is saved as it arrives, so an interrupted run resumes instead of paying twice.

Checks before handover

- every document produced an answer, and failures are recorded rather than dropped
- replies that could not be parsed stay under a set share
- the spread of answers matches the labeled sample
- blanks are not far above what the validation set showed

Documents are sent for review for five reasons, one of which is a random sample. The other four select documents that are already doubtful, so a quality estimate from those alone reads worse than the run is.

Testing the comparison code

If the comparison is wrong, every number after it is wrong, and the search spends money working against it.

- A generated corpus, contracts in this case, where a plausible wrong answer sits near the right one: a former company name, a second dollar figure, a notary in the signature block, a venue clause naming a different state from the governing-law clause.
- Each case lists the forms a correct answer might take, which the comparison must accept, and the near-miss it must reject.
- It runs in the test suite without model calls. It found four faults in the comparison code: dotted legal suffixes, titles on personal names, currency written as a word, and a typed answer that skipped unit handling.
- One limit is recorded rather than fixed. String similarity cannot separate a typo from a different company, so "Smith & Hartley Ltd." is accepted for "Smith and Hart Limited", and borderline cases go to review.

The commands

COMMAND	WHAT IT DOES
<code>newproject</code>	Create the project, pinned to one version of the package
<code>extract</code>	PDFs to cached text, scanned pages transcribed
<code>sample-labels</code> , <code>label-sheet</code> , <code>import-labels</code>	Choose what to label, produce the spreadsheet, read it back
<code>audit-labels</code>	Check labels against the questions and count examples per answer
<code>make-splits</code>	Divide the labeled documents; stop where an answer is too rare to measure
<code>run-baseline</code>	Score simple versions, so tuning has something to beat
<code>compile</code>	One tuning experiment, changing one thing
<code>holdout</code>	Score the held-back documents, once
<code>production</code>	Run the whole corpus and check the output
<code>rescore</code> , <code>adjudicate</code>	Re-score a finished run at no cost; list the borderline matches
<code>close</code>	Write the report, the shipped prompt, and the ledger row

`status` works out where a project stands from what is on disk, so it is correct in a new session. Each phase has a guidance file that a Claude Code session reads before running it.

What a finished project hands over

FILE	CONTENTS
<code>REPORT.md</code>	Written for people who did not build the project: what it does, how well it works, what still needs a person. Each number links to the file it came from
<code>PROMPT.md</code>	The prompt that shipped, in readable form: instructions, the question per task, the worked examples
<code>outputs.jsonl</code>	The answers for every document, including failures
<code>qa_report.md</code>	The checks on the full run and the documents queued for review
<code>decisions.md</code>	Each decision a person made, with the numbers it was made on
<code>spend.json</code>	What each paid step cost
<code>ledger.csv</code>	One row per project, so the next one can be estimated from it

The report is regenerated each time `close` runs. Anything written by hand goes in `REPORT_NOTES.md`, which is appended to it.

What the measurement does not cover

- Small sets give wide ranges. Thirty documents cannot settle a question to within a few points, and the report gives the range rather than implying precision.
- Rare answers are reported as unmeasured. At 2% prevalence, reaching thirty examples by random sampling takes about 1,500 documents. The options are to label more of that kind, merge it with a neighbouring answer, or leave it unmeasured, and that choice is recorded.
- Answers come from text. Scanned pages are transcribed first, so page layout is not available to the model when it answers.
- A labeling mistake looks the same as a model mistake. Reading the failures by hand is part of the work.
- Running a real project found fourteen bugs in the package, one of which paid for a batch and then paid again for live requests. All are fixed, and each has a test.

Starting a project

```
newproject "Supplier Agreements"    # creates the project, pinned to one version
cd supplier-agreements && make lock && make sync
claude                             # from inside the project folder
```

Claude then runs the commands, explains what comes back, and stops at the decisions that are yours.

- Yours to decide: how many documents to label, whether the spreadsheet arrives filled in, what to do about an answer with too few examples, whether to ship the tuned prompt or the simple one, and when to score the held-back documents.
- Not Claude's to do: the labels, and in particular the held-back ones. Labels written by a model are not human labels, and there they would measure the model against itself.

The project's `README.md` and `docs/protocol.md` explain each step.